Description Submission view

Due date: Cumartesi, 17 Mayıs 2025, 11:59 PM
Requested files: MainProgram.java (Download)
Type of work: Individual work

Advanced Digital Media Library Management System

Project Objective: In this assignment, you are expected to develop a comprehensive digital media library management system. The system should be capable of managing different types of media items (books, movies, music albums) and members, as well as tracking borrowing and return operations.

Mandatory Requirements:

1. `Borrowable` **Interface:**
   - This interface will define the behaviors for a media item that can be borrowed and returned.
   - **Methods:**
     - `void borrowItem()`
     - `void returnItem()`
     - `String getStatus()`
2. `Media` **Abstract Class:**
   - This class will contain common attributes and behaviors for all media types.
   - **Attributes:**
     - `id` (String, unique identifier for each media item)
     - `title` (String)
     - `publicationYear` (int)
   - **Methods:**
     - Parameterized `constructor`.
     - `getter` and `setter` methods for all attributes (Encapsulation).
     - `abstract String getDetails()`: An abstract method that subclasses must implement to return their specific details.
     - A `toString()` method providing basic information about the media (subclasses can extend this).
3. `Media` **Subclasses (Inheritance and Polymorphism):**
   - The following classes will inherit from the `Media` abstract class and implement the `Borrowable` interface. Each of these classes should also include a `private String status;` field, typically initialized to "Available" in their constructor.
   - **a.** `Book` **Class (extends** `Media` **implements** `Borrowable`**):**
     - **Additional Attributes:** `author` (String), `isbn` (String), `pageCount` (int).
     - **Methods:**
       - A `constructor` that calls the `Media` class constructor and initializes its own specific attributes.
       - Implementation of `borrowItem()`, `returnItem()`, and `getStatus()` methods from the `Borrowable` interface.
       - Override of the `getDetails()` abstract method from the `Media` class to include book-specific details (author, ISBN, page count).
       - Override `toString()` if necessary for a more detailed representation.
   - **b.** `Movie` **Class (extends** `Media` **implements** `Borrowable`**):**
     - **Additional Attributes:** `director` (String), `durationMinutes` (int), `genre` (String, e.g., "Sci-Fi", "Drama").
     - **Methods:**
       - `Constructor`.
       - Implementation of `Borrowable` methods.
       - Override of `getDetails()` to include movie-specific details.
       - Override `toString()` if necessary.
   - **c.** `MusicAlbum` **Class (extends** `Media` **implements** `Borrowable`**):**
     - **Additional Attributes:** `artist` (String), `trackCount` (int), `albumGenre` (String, e.g., "Pop", "Rock").
     - **Methods:**
       - `Constructor`.

?

        - Implementation of `Borrowable` methods.
        - Override of `getDetails()` to include music album-specific details.
        - Override `toString()` if necessary.
4. `Member` **Class:**
    - **Attributes:**
        - `memberId` (String, unique identifier)
        - `firstName` (String)
        - `lastName` (String)
        - `borrowedMediaItems` (`ArrayList<Borrowable>`, stores the list of various media items currently borrowed by the member. The `Borrowable` interface type will be used here.)
    - **Methods:**
        - Parameterized `constructor`.
        - `getter` and `setter` methods.
        - `void borrowMedia(Borrowable media)`: Adds a media item to the member's list of borrowed items.
        - `void returnMedia(Borrowable media)`: Removes a media item from the member's list of borrowed items.
        - A `toString()` method that displays member information and the list of borrowed media items (if any).
5. **Custom Exception Classes:**
    - Define your own exception classes for the following situations (all can extend `Exception` or an appropriate subclass):
        - `MediaNotFoundException`: Thrown when a searched media item is not found in the library.
        - `MemberNotFoundException`: Thrown when a searched member is not found in the system.
        - `CannotBorrowException`: Thrown when a media item cannot be borrowed (e.g., already borrowed, not in stock).
        - `CannotReturnException`: Thrown when a media item cannot be returned (e.g., not borrowed by this member).
        - `MediaAlreadyExistsException`: Thrown if a media item being added (with the same ID) already exists in the library.
        - `MemberAlreadyExistsException`: Thrown if a member being added (with the same ID) is already registered in the system.
6. `LibraryManagement` **Class:**
    - This is the central class that will manage all library operations.
    - **Attributes:**
        - `mediaCollection` (`HashMap<String, Media>`, Media ID as key, `Media` object as value. This allows storing different media types together – Polymorphism.)
        - `memberList` (`HashMap<String, Member>`, Member ID as key, `Member` object as value.)
    - **Methods (These methods should throw the custom exceptions defined above where appropriate):**
        - `void addMedia(Media mediaItem) throws MediaAlreadyExistsException`
        - `void removeMedia(String mediaId) throws MediaNotFoundException`
        - `Media findMediaById(String mediaId) throws MediaNotFoundException`
        - `ArrayList<Media> findMediaByTitle(String title)`
        - `void addMember(Member member) throws MemberAlreadyExistsException`
        - `void removeMember(String memberId) throws MemberNotFoundException`
        - `Member findMemberById(String memberId) throws MemberNotFoundException`
        - `void processBorrow(String mediaId, String memberId) throws MediaNotFoundException, MemberNotFoundException, CannotBorrowException`
        - `void processReturn(String mediaId, String memberId) throws MediaNotFoundException, MemberNotFoundException, CannotReturnException`
        - `void listAllMedia()`: Lists all media items with their details using their `getDetails()` methods (Polymorphism).
        - `void listAllMembers()`
        - `void listMemberBorrowedItems(String memberId) throws MemberNotFoundException`

Detailed Method Explanations:

1. `Borrowable` Interface

This interface defines the contract for objects that can be borrowed and returned.

- `void borrowItem()`
    - **What it does:** Called when an instance of a class implementing this method (a media item) is borrowed. It should update the item's status to "Borrowed" and, if applicable, decrease any stock count.
    - **What it returns:** `void`.
    - **Parameters it takes:** None.
- `void returnItem()`
    - **What it does:** Called when an instance of a class implementing this method (a media item) is returned to the library. It should update the item's status to "Available" and, if applicable, increase any stock count.
    - **What it returns:** `void`.

    - **Parameters it takes:** None.
- `String getStatus()`
    - **What it does:** Returns the current borrowing status (e.g., "Available", "Borrowed") of an instance of a class implementing this method.
    - **What it returns:** `String` - The current status of the media item.
    - **Parameters it takes:** None.

---

2. `Media` Abstract Class

Defines common properties and behaviors for all media types. Cannot be instantiated directly.

- `Media(String id, String title, int publicationYear)` **(Constructor)**
    - **What it does:** Called when a new `Media` object is created (via its subclasses). Initializes its core attributes.
    - **What it returns:** N/A (constructor).
    - **Parameters it takes:**
        - `id` (Type: `String`): The unique identifier of the media item.
        - `title` (Type: `String`): The title of the media item.
        - `publicationYear` (Type: `int`): The publication year of the media item.
- `String getId()`
    - **What it does:** Returns the unique identifier (`id`) of the media item.
    - **What it returns:** `String` - The ID of the media item.
    - **Parameters it takes:** None.
- `void setId(String id)`
    - **What it does:** Sets the unique identifier (`id`) of the media item.
    - **What it returns:** `void`.
    - **Parameters it takes:**
        - `id` (Type: `String`): The new ID to be set.
- `String getTitle()`
    - **What it does:** Returns the title of the media item.
    - **What it returns:** `String` - The title of the media item.
    - **Parameters it takes:** None.
- `void setTitle(String title)`
    - **What it does:** Sets the title of the media item.
    - **What it returns:** `void`.
    - **Parameters it takes:**
        - `title` (Type: `String`): The new title to be set.
- `int getPublicationYear()`
    - **What it does:** Returns the publication year of the media item.
    - **What it returns:** `int` - The publication year of the media item.
    - **Parameters it takes:** None.
- `void setPublicationYear(int publicationYear)`
    - **What it does:** Sets the publication year of the media item.
    - **What it returns:** `void`.
    - **Parameters it takes:**
        - `publicationYear` (Type: `int`): The new publication year to be set.
- `abstract String getDetails()`
    - **What it does:** This is an abstract method. Each subclass inheriting from `Media` must implement (override) this method to return details specific to that media type (e.g., author and ISBN for a book, director and duration for a movie).
    - **What it returns:** `String` - Detailed information specific to the media item.
    - **Parameters it takes:** None.
- `String toString()`
    - **What it does:** Returns a string representation of the media item, typically including basic information like ID, title, and publication year. Subclasses can override this method to add more details.
    - **What it returns:** `String` - A string representation of the media item.
    - **Parameters it takes:** None.

---

3. `Media` Subclasses

These classes inherit from `Media` and implement `Borrowable`. Each should have a `private String status;` field, typically initialized to `"Available"`.

a. `Book` Class (extends `Media` implements `Borrowable`)

- **Additional Attributes:** `author` (String), `isbn` (String), `pageCount` (int), `status` (String).
- **`Book(String id, String title, int publicationYear, String author, String isbn, int pageCount)` (Constructor)**
  - **What it does:** Creates a new `Book` object. Calls the `Media` (superclass) constructor to initialize common attributes (`id`, `title`, `publicationYear`) and initializes book-specific attributes (`author`, `isbn`, `pageCount`). Initializes `status` to "Available".
  - **What it returns:** N/A (constructor).
  - **Parameters it takes:**
    - `id`, `title`, `publicationYear`: For the `Media` constructor.
    - `author` (Type: `String`): The author of the book.
    - `isbn` (Type: `String`): The ISBN of the book.
    - `pageCount` (Type: `int`): The page count of the book.
- **Getter/Setter Methods:** Standard getters and setters for `author`, `isbn`, `pageCount`.
- **`void borrowItem()` (implements `Borrowable`)**
  - **What it does:** Sets the book's `status` attribute to "Borrowed". Manages stock if applicable (e.g., if an `availableCopies` attribute exists).
  - **What it returns:** `void`.
  - **Parameters it takes:** None.
- **`void returnItem()` (implements `Borrowable`)**
  - **What it does:** Sets the book's `status` attribute to "Available".
  - **What it returns:** `void`.
  - **Parameters it takes:** None.
- **`String getStatus()` (implements `Borrowable`)**
  - **What it does:** Returns the current `status` of the book.
  - **What it returns:** `String` - The status of the book.
  - **Parameters it takes:** None.
- **`String getDetails()` (overrides `Media.getDetails()`)**
  - **What it does:** Returns a string combining basic media information from the superclass with book-specific details (author, ISBN, page count).
  - **What it returns:** `String` - Detailed information about the book.
  - **Parameters it takes:** None.
- **`String toString()` (overrides `Media.toString()`, optional but recommended)**
  - **What it does:** Returns a more comprehensive string representation of the `Book` object (e.g., ID, title, author). Often similar to or uses `getDetails()`.
  - **What it returns:** `String`.
  - **Parameters it takes:** None.

b. `Movie` Class (extends `Media` implements `Borrowable`)

- **Additional Attributes:** `director` (String), `durationMinutes` (int), `genre` (String), `status` (String).
- **`Movie(String id, String title, int publicationYear, String director, int durationMinutes, String genre)` (Constructor)**
  - **What it does:** Creates a new `Movie` object. Initializes `Media` attributes and movie-specific attributes. Initializes `status` to "Available".
  - **What it returns:** N/A.
  - **Parameters it takes:**
    - `id`, `title`, `publicationYear`: For `Media`.
    - `director` (Type: `String`): The director of the movie.
    - `durationMinutes` (Type: `int`): The duration of the movie in minutes.
    - `genre` (Type: `String`): The genre of the movie.
- **Getter/Setter Methods:** For `director`, `durationMinutes`, `genre`.
- **`borrowItem()`, `returnItem()`, `getStatus()`:** Implemented similarly to the `Book` class, updating the `status` attribute.
- **`String getDetails()` (overrides `Media.getDetails()`)**
  - **What it does:** Returns a string with movie-specific details (director, duration, genre) and basic information.
  - **What it returns:** `String` - Detailed information about the movie.
  - **Parameters it takes:** None.
- **`String toString()`:** Similar to the `Book` class.

c. `MusicAlbum` Class (extends `Media` implements `Borrowable`)

- **Additional Attributes:** `artist` (String), `trackCount` (int), `albumGenre` (String), `status` (String).
- **`MusicAlbum(String id, String title, int publicationYear, String artist, int trackCount, String albumGenre)` (Constructor)**
  - **What it does:** Creates a new `MusicAlbum` object. Initializes `Media` attributes and album-specific attributes. Initializes `status` to "Available".
  - **What it returns:** N/A.
  - **Parameters it takes:**
    - `id`, `title`, `publicationYear`: For `Media`.

        - `artist` (Type: `String`): The artist of the album.
        - `trackCount` (Type: `int`): The number of tracks in the album.
        - `albumGenre` (Type: `String`): The genre of the album.
- **Getter/Setter Methods:** For `artist`, `trackCount`, `albumGenre`.
- **`borrowItem()`, `returnItem()`, `getStatus()`:** Implemented similarly to the `Book` class, updating the `status` attribute.
- **`String getDetails()` (overrides `Media.getDetails()`)**
    - **What it does:** Returns a string with music album-specific details (artist, track count, album genre) and basic information.
    - **What it returns:** `String` - Detailed information about the music album.
    - **Parameters it takes:** None.
- **`String toString()`:** Similar to the `Book` class.

---

4. `Member` Class

Represents library members.

- **`Member(String memberId, String firstName, String lastName)` (Constructor)**
    - **What it does:** Creates a new `Member` object and initializes its attributes. Initializes `borrowedMediaItems` as an empty `ArrayList`.
    - **What it returns:** N/A.
    - **Parameters it takes:**
        - `memberId` (Type: `String`): The unique identifier of the member.
        - `firstName` (Type: `String`): The first name of the member.
        - `lastName` (Type: `String`): The last name of the member.
- **Getter/Setter Methods:** For `memberId`, `firstName`, `lastName`.
- **`ArrayList<Borrowable> getBorrowedMediaItems()`**
    - **What it does:** Returns the list of media items currently borrowed by the member.
    - **What it returns:** `ArrayList<Borrowable>` - The list of borrowed media items.
    - **Parameters it takes:** None.
- **`void borrowMedia(Borrowable media)`**
    - **What it does:** Adds the given `Borrowable` media item to the member's `borrowedMediaItems` list.
    - **What it returns:** `void`.
    - **Parameters it takes:**
        - `media` (Type: `Borrowable`): The media item being borrowed by the member.
- **`void returnMedia(Borrowable media)`**
    - **What it does:** Removes the given `Borrowable` media item from the member's `borrowedMediaItems` list.
    - **What it returns:** `void`.
    - **Parameters it takes:**
        - `media` (Type: `Borrowable`): The media item being returned by the member.
- **`String toString()`**
    - **What it does:** Returns a string representation of the member's information (ID, first name, last name) and a list of their borrowed media items (if any).
    - **What it returns:** `String` - Member information.
    - **Parameters it takes:** None.

---

5. Custom Exception Classes

These classes extend `Exception` (or a suitable subclass) and represent specific error conditions. They typically have a constructor that accepts a message string.

- `MediaNotFoundException(String message)`
- `MemberNotFoundException(String message)`
- `CannotBorrowException(String message)`
- `CannotReturnException(String message)`
- `MediaAlreadyExistsException(String message)`
- `MemberAlreadyExistsException(String message)`

---

6. `LibraryManagement` Class

The central class for managing all library media and member operations.

- **`LibraryManagement()` (Constructor)**

    - **What it does:** Initializes the `mediaCollection` (`HashMap<String, Media>`) and `memberList` (`HashMap<String, Member>`) as empty collections.
    - **What it returns:** N/A.
    - **Parameters it takes:** None.

- `void addMedia(Media mediaItem) throws MediaAlreadyExistsException`
  - **What it does:** Adds the given `Media` object to the `mediaCollection`. Before adding, it checks if a media item with the same ID already exists. If so, it throws `MediaAlreadyExistsException`.
  - **What it returns:** `void`.
  - **Parameters it takes:**
    - `mediaItem` (Type: `Media`): The media object to be added.
  - **Exceptions it may throw:** `MediaAlreadyExistsException`.
- `void removeMedia(String mediaId) throws MediaNotFoundException`
  - **What it does:** Removes the media item with the given `mediaId` from the `mediaCollection`. If the media item is not found, it throws `MediaNotFoundException`. *Note: How to handle deletion of a borrowed media item (e.g., prevent it, force return) should be decided based on project requirements. This definition only covers the not-found case.*
  - **What it returns:** `void`.
  - **Parameters it takes:**
    - `mediaId` (Type: `String`): The ID of the media to be removed.
  - **Exceptions it may throw:** `MediaNotFoundException`.
- `Media findMediaById(String mediaId) throws MediaNotFoundException`
  - **What it does:** Searches for a media item by the given `mediaId` in the `mediaCollection`. If found, it returns the `Media` object. If not found, it throws `MediaNotFoundException`.
  - **What it returns:** `Media` - The found media object.
  - **Parameters it takes:**
    - `mediaId` (Type: `String`): The ID of the media to search for.
  - **Exceptions it may throw:** `MediaNotFoundException`.
- `ArrayList<Media> findMediaByTitle(String title)`
  - **What it does:** Searches for media items in `mediaCollection` whose titles contain the given `title` string (case-insensitive search is good practice). Returns an `ArrayList` of all matching `Media` objects. Returns an empty list if no matches are found.
  - **What it returns:** `ArrayList<Media>` - A list of matching media items.
  - **Parameters it takes:**
    - `title` (Type: `String`): The title or part of the title to search for.
- `void addMember(Member member) throws MemberAlreadyExistsException`
  - **What it does:** Adds the given `Member` object to the `memberList`. Before adding, it checks if a member with the same ID is already registered. If so, it throws `MemberAlreadyExistsException`.
  - **What it returns:** `void`.
  - **Parameters it takes:**
    - `member` (Type: `Member`): The member object to be added.
  - **Exceptions it may throw:** `MemberAlreadyExistsException`.
- `void removeMember(String memberId) throws MemberNotFoundException`
  - **What it does:** Removes the member with the given `memberId` from the `memberList`. If the member is not found, it throws `MemberNotFoundException`. *Note: How to handle deletion if the member has borrowed items (e.g., prevent deletion, force return of items) depends on project design. This definition only covers the not-found case.*
  - **What it returns:** `void`.
  - **Parameters it takes:**
    - `memberId` (Type: `String`): The ID of the member to be removed.
  - **Exceptions it may throw:** `MemberNotFoundException`.
- `Member findMemberById(String memberId) throws MemberNotFoundException`
  - **What it does:** Searches for a member by the given `memberId` in the `memberList`. If found, it returns the `Member` object. If not found, it throws `MemberNotFoundException`.
  - **What it returns:** `Member` - The found member object.
  - **Parameters it takes:**
    - `memberId` (Type: `String`): The ID of the member to search for.
  - **Exceptions it may throw:** `MemberNotFoundException`.
- `void processBorrow(String mediaId, String memberId) throws MediaNotFoundException, MemberNotFoundException, CannotBorrowException`
  - **What it does:** Processes the borrowing of a media item by a member.
    1. Finds the media item by `mediaId` (throws `MediaNotFoundException` if not found).
    2. Finds the member by `memberId` (throws `MemberNotFoundException` if not found).
    3. Checks if the media item implements `Borrowable` (via type casting).
    4. Checks if the media item can be borrowed using its `getStatus()` method (must be "Available"). If not, throws `CannotBorrowException`.
    5. Calls the `borrowItem()` method on the media item.

        6. Calls the `borrowMedia()` method on the member to add the media to their borrowed list.
    - **What it returns:** `void`.
    - **Parameters it takes:**
        - `mediaId` (Type: `String`): The ID of the media item to be borrowed.
        - `memberId` (Type: `String`): The ID of the member borrowing the media.
    - **Exceptions it may throw:** `MediaNotFoundException`, `MemberNotFoundException`, `CannotBorrowException`.
- **`void processReturn(String mediaId, String memberId) throws MediaNotFoundException, MemberNotFoundException, CannotReturnException`**
    - **What it does:** Processes the return of a media item by a member.
        1. Finds the media item by `mediaId` (throws `MediaNotFoundException` if not found).
        2. Finds the member by `memberId` (throws `MemberNotFoundException` if not found).
        3. Checks if the media item implements `Borrowable`.
        4. Checks if the media item is in the member's borrowed list and if its status is "Borrowed". If not appropriate, throws `CannotReturnException`.
        5. Calls the `returnItem()` method on the media item.
        6. Calls the `returnMedia()` method on the member to remove the media from their borrowed list.
    - **What it returns:** `void`.
    - **Parameters it takes:**
        - `mediaId` (Type: `String`): The ID of the media item to be returned.
        - `memberId` (Type: `String`): The ID of the member returning the media.
    - **Exceptions it may throw:** `MediaNotFoundException`, `MemberNotFoundException`, `CannotReturnException`.
- **`void listAllMedia()`**
    - **What it does:** Lists all media items in the `mediaCollection`. For each media item, it prints its detailed information using its `getDetails()` (or `toString()`) method (demonstrating Polymorphism).
    - **What it returns:** `void`.
    - **Parameters it takes:** None.
- **`void listAllMembers()`**
    - **What it does:** Lists all members in the `memberList`. For each member, it prints their information using their `toString()` method.
    - **What it returns:** `void`.
    - **Parameters it takes:** None.
- **`void listMemberBorrowedItems(String memberId) throws MemberNotFoundException`**
    - **What it does:** Finds the member with the given `memberId` (throws `MemberNotFoundException` if not found). Then, it prints the list of media items borrowed by that member using the member's `getBorrowedMediaItems()` method.
    - **What it returns:** `void`.
    - **Parameters it takes:**
        - `memberId` (Type: `String`): The ID of the member whose borrowed items are to be listed.
    - **Exceptions it may throw:** `MemberNotFoundException`.

Requested files

MainProgram.java

```java
public class MainProgram {
    public static void main(String[] args) {
        LibraryManagement library = new LibraryManagement();
        System.out.println("--- Library Management System Test Suite ---");
        System.out.println("--- Current Date: " + java.time.LocalDate.now() + " ---\n");


        // Test 1: Adding Media Items
        System.out.println("--- Test 1: Adding Media Items ---");
        try {
            Book book1 = new Book("B001", "The Lord of the Rings", 1954, "J.R.R. Tolkien", "0618260274", 1178);
            Movie movie1 = new Movie("M001", "Inception", 2010, "Christopher Nolan", 148, "Sci-Fi");
            MusicAlbum album1 = new MusicAlbum("A001", "Dark Side of the Moon", 1973, "Pink Floyd", 10, "Progressive Rock");
            Book book2 = new Book("B002", "1984", 1949, "George Orwell", "9780451524935", 328);

            library.addMedia(book1);
            library.addMedia(movie1);
            library.addMedia(album1);
            library.addMedia(book2);

            System.out.println("\n--- Attempting to add existing media (B001), expecting MediaAlreadyExistsException ---");
            library.addMedia(new Book("B001", "Duplicate Book", 2000, "Test Author", "123", 100)); // Should throw exception
        } catch (MediaAlreadyExistsException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (Exception e) {
            System.err.println("UNEXPECTED EXCEPTION during media addition: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }
        library.listAllMedia();

        // Test 2: Adding Members
        System.out.println("\n--- Test 2: Adding Members ---");
        try {
            Member member1 = new Member("MEM001", "John", "Doe");
            Member member2 = new Member("MEM002", "Jane", "Smith");
            library.addMember(member1);
            library.addMember(member2);

            System.out.println("\n--- Attempting to add existing member (MEM001), expecting MemberAlreadyExistsException ---");
            library.addMember(new Member("MEM001", "Johnny", "Doeman")); // Should throw exception
        } catch (MemberAlreadyExistsException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (Exception e) {
            System.err.println("UNEXPECTED EXCEPTION during member addition: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }
        library.listAllMembers();

        // Test 3: Finding Media by ID
        System.out.println("\n--- Test 3: Finding Media by ID ---");
        try {
            System.out.println("Finding B001: " + library.findMediaById("B001").getDetails());
            System.out.println("Finding M001: " + library.findMediaById("M001").getDetails());
            System.out.println("\n--- Attempting to find non-existent media (X999), expecting MediaNotFoundException ---");
            library.findMediaById("X999"); // Should throw exception
        } catch (MediaNotFoundException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (Exception e) {
            System.err.println("UNEXPECTED EXCEPTION during findMediaById: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }

        // Test 4: Finding Member by ID
        System.out.println("\n--- Test 4: Finding Member by ID ---");
        try {
            System.out.println("Finding MEM001: " + library.findMemberById("MEM001").getFirstName());
            System.out.println("\n--- Attempting to find non-existent member (XMEM9), expecting MemberNotFoundException ---");
            library.findMemberById("XMEM9"); // Should throw exception
        } catch (MemberNotFoundException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (Exception e) {
            System.err.println("UNEXPECTED EXCEPTION during findMemberById: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }

        // Test 5: Finding Media by Title
        System.out.println("\n--- Test 5: Finding Media by Title ---");
        System.out.println("Searching for 'Lord':");
        List<Media> foundByTitle = library.findMediaByTitle("Lord");
        if (foundByTitle.isEmpty()) System.out.println("Info: No media found with title containing 'Lord'.");
        else foundByTitle.forEach(m -> System.out.println("- " + m.getDetails()));

        System.out.println("\nSearching for 'NonExistentTitle':");
        foundByTitle = library.findMediaByTitle("NonExistentTitle");
        if (foundByTitle.isEmpty()) System.out.println("Info: No media found with title containing 'NonExistentTitle'.");
        else foundByTitle.forEach(m -> System.out.println("- " + m.getDetails()));


        // Test 6: Processing Borrows
        System.out.println("\n--- Test 6: Processing Borrows ---");
        try {
            System.out.println("MEM001 attempting to borrow B001 (The Lord of the Rings):");
            library.processBorrow("B001", "MEM001");
            library.listMemberBorrowedItems("MEM001");
            // Check status directly from the retrieved media object
            Media borrowedBook1 = library.findMediaById("B001");
            if (borrowedBook1 instanceof Borrowable) {
                 System.out.println("Status of B001 after borrow: " + ((Borrowable) borrowedBook1).getStatus());
            }


            System.out.println("\nMEM002 attempting to borrow M001 (Inception):");
            library.processBorrow("M001", "MEM002");
            library.listMemberBorrowedItems("MEM002");

            System.out.println("\n--- Attempting to borrow already borrowed item (B001 by MEM002), expecting CannotBorrowException ---");
            library.processBorrow("B001", "MEM002"); // Should throw exception
```

```
        } catch (CannotBorrowException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (MediaNotFoundException | MemberNotFoundException e) {
            System.err.println("UNEXPECTED ERROR during borrow tests: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }

        try {
            System.out.println("\n--- Attempting to borrow non-existent media (X999 by MEM001), expecting MediaNotFoundException ---");
            library.processBorrow("X999", "MEM001"); // Should throw exception
        } catch (MediaNotFoundException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (MemberNotFoundException | CannotBorrowException e) {
            System.err.println("UNEXPECTED ERROR during borrow non-existent media test: " + e.getClass().getSimpleName() + " - " + e.getMes
        }

        try {
            System.out.println("\n--- Attempting to borrow by non-existent member (B002 by XMEM9), expecting MemberNotFoundException ---");
            library.processBorrow("B002", "XMEM9"); // Should throw exception
        } catch (MemberNotFoundException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (MediaNotFoundException | CannotBorrowException e) {
            System.err.println("UNEXPECTED ERROR during borrow by non-existent member test: " + e.getClass().getSimpleName() + " - " + e.ge
        }

        // Test 7: Processing Returns
        System.out.println("\n--- Test 7: Processing Returns ---");
        try {
            System.out.println("MEM001 attempting to return B001 (The Lord of the Rings):");
            library.processReturn("B001", "MEM001");
            library.listMemberBorrowedItems("MEM001");
            Media returnedBook1 = library.findMediaById("B001");
            if (returnedBook1 instanceof Borrowable) {
                 System.out.println("Status of B001 after return: " + ((Borrowable) returnedBook1).getStatus());
            }

            System.out.println("\n--- Attempting to return item not borrowed by member (M001 by MEM001), expecting CannotReturnException --
            library.processReturn("M001", "MEM001"); // Should throw exception
        } catch (CannotReturnException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (MediaNotFoundException | MemberNotFoundException e) {
            System.err.println("UNEXPECTED ERROR during return tests: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }

        try {
            System.out.println("\n--- Attempting to return an already available/returned item (B001 by MEM001 again), expecting CannotRetur
            library.processReturn("B001", "MEM001"); // Should throw exception
        } catch (CannotReturnException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (MediaNotFoundException | MemberNotFoundException e) {
            System.err.println("UNEXPECTED ERROR during return already available item test: " + e.getClass().getSimpleName() + " - " + e.ge
        }

        try {
            System.out.println("\n--- MEM002 returns M001 (Inception) ---");
            library.processReturn("M001", "MEM002");
            library.listMemberBorrowedItems("MEM002");
        } catch (Exception e) { // Catch generic Exception for unexpected issues in this valid step
             System.err.println("UNEXPECTED ERROR during valid return of M001 by MEM002: " + e.getClass().getSimpleName() + " - " + e.getMe
        }


        // Test 8: Removing Media
        System.out.println("\n--- Test 8: Removing Media ---");
        try {
            System.out.println("Removing A001 (Dark Side of the Moon):");
            library.removeMedia("A001"); // This was added and not borrowed
            library.listAllMedia();

            System.out.println("\n--- Attempting to remove non-existent media (X999), expecting MediaNotFoundException ---");
            library.removeMedia("X999"); // Should throw exception
        } catch (MediaNotFoundException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (Exception e) {
            System.err.println("UNEXPECTED EXCEPTION during media removal: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }

        System.out.println("\n--- Test 8.1: Attempting to remove a borrowed item and its consequences ---");
        try {
            System.out.println("MEM001 borrows B002 (1984).");
            library.processBorrow("B002", "MEM001");
            library.listMemberBorrowedItems("MEM001");
            System.out.println("Attempting to remove B002 (which is now borrowed by MEM001)...");
            library.removeMedia("B002");
            System.out.println("B002 removed. Current state of media and member MEM001's borrowed items:");
            library.listAllMedia();
            library.listMemberBorrowedItems("MEM001"); // B002 should be gone from here too if removeMedia handled it
        } catch (Exception e) {
            System.err.println("EXCEPTION during borrow or removal of B002: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }


        // Test 9: Removing Members
        System.out.println("\n--- Test 9: Removing Members ---");
        try {
            System.out.println("MEM001 borrows B001 again for member removal test.");
            library.processBorrow("B001", "MEM001");
        } catch (Exception e) {
            System.err.println("Error during pre-setup for member removal test (borrowing B001): " + e.getMessage());
        }

        try {
            System.out.println("Attempting to remove member MEM001 (John Doe) who has borrowed B001:");
            library.removeMember("MEM001");
```

```java
            library.listAllMembers();
            Media bookB001 = library.findMediaById("B001"); // This will fail if B001 was borrowed by removed member and not auto-returned
            if (bookB001 instanceof Borrowable) {
                System.out.println("Status of B001 after MEM001 (borrower) was removed: " + ((Borrowable)bookB001).getStatus());
            }


            System.out.println("\nRemoving member MEM002 (Jane Smith - no borrowed items):");
            library.removeMember("MEM002");
            library.listAllMembers();

            System.out.println("\n--- Attempting to remove non-existent member (XMEM9), expecting MemberNotFoundException ---");
            library.removeMember("XMEM9"); // Should throw exception
        } catch (MemberNotFoundException e) {
            System.err.println("CAUGHT EXPECTED EXCEPTION: " + e.getMessage());
        } catch (Exception e) {
            System.err.println("UNEXPECTED EXCEPTION during member removal: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }

        // Test 10: Listing with Empty or Near-Empty Collections
        System.out.println("\n--- Test 10: Listing with Empty/Near-Empty Collections ---");
        try {
            System.out.println("Removing any remaining media and members for empty list tests...");
            if (mediaCollectionContains(library, "B001")) {
                Media b001 = library.findMediaById("B001");
                if (b001 instanceof Borrowable && ((Borrowable)b001).getStatus().equals("Borrowed")) {
                    System.out.println("Note: B001 is still 'Borrowed' despite member removal. Manually setting to Available for cleanup.")
                    ((Borrowable)b001).returnItem(); // Manually make it available if necessary for other tests
                }
                library.removeMedia("B001");
            }
            if (mediaCollectionContains(library, "M001")) library.removeMedia("M001");

        } catch (Exception e) {
            System.err.println("Error during cleanup for empty list tests: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }
        library.listAllMedia();
        library.listAllMembers();


        System.out.println("\n--- Test Suite Completed ---");
    }

    // Helper method to check if media exists without throwing an exception internally for the test suite
    private static boolean mediaCollectionContains(LibraryManagement lib, String mediaId) {
        try {
            lib.findMediaById(mediaId);
            return true;
        } catch (MediaNotFoundException e) {
            return false;
        }
    }
}
```

VPL